Executive Briefing: Choosing Versus Accepting: Managing the Hidden Technical Debt of AI Coding Assistants

The Key Problem

- AI coding assistants have made working code far cheaper to produce, yet early evidence shows code complexity, duplication, and persistent defects rising in the codebases that adopt them.
- Digital leaders typically judge assistants by delivery speed and defect counts and manage technical debt with registers and repayment sprints. These tools assume that debt is visible and that teams repay it when the numbers justify doing so.

Key Insights

Based on a formal model of how assistants change developers' quality decisions, and on an analysis of 32,620 commits in five open-source projects:

- Assistants change the order of the decision. Developers used to decide how to build a task before writing it; now working code arrives first, and the decision becomes whether to accept it. Well-known biases act only in this second situation.
- Rising total debt does not show that anything is wrong with how developers decide, because faster work alone can produce the same rise. Three other measures can tell the difference: how much debt developers flag, how often they defer refactoring when working code is in hand, and how often planned refactoring never happens.
- A more reliable model can leave more debt, because developers stop reviewing generated code faster than its error rate falls.
- A codebase can tip into a high-debt state that small nudges cannot reverse, and a temporary review mandate can have effects that outlast it.

What Executives Should Do

- Replace total debt with the three behavioral measures when evaluating assistants.
- Ask developers to commit to an approach before generating code for consequential changes.
- Evaluate every model upgrade by how much review it displaces, not only by its error rate.
- Size review mandates to correct developers' beliefs about generated code, not to control them.
- Identify codebases at risk of tipping and fund one-time refactoring for those that do.

Choosing Versus Accepting: Managing the Hidden Technical Debt of AI Coding Assistants

AI coding assistants deliver working code before developers have decided how a task should be built, turning a choice into an acceptance decision. Drawing on a formal model and commit data from five open-source projects, we show why total technical debt is a misleading scoreboard for these tools, why better models can leave more debt, and why short review mandates can have lasting effects. We recommend five practices that help digital leaders evaluate and govern AI-assisted software development.

Keywords

AI coding assistants, technical debt, software governance, behavioral economics, code review

The Puzzle: Cheaper Code, Costlier Systems

Few tools have spread through software organizations as quickly as AI coding assistants, and their promise is easy to see: developers produce far more working code per hour. What happens to that code afterward is less encouraging. In open-source projects that adopted an agentic assistant, development velocity rose only briefly, while static-analysis warnings and code complexity rose persistently.[1] An audit of code produced by AI models and coding agents found that more capable models generate more bloated and tightly coupled code,[2] and a study of about 303,000 AI-authored commits found that 22.7% of the issues they introduced were still present in the latest version of the code.[3]

These findings are puzzling. Technical debt has long been understood as a deliberate trade-off: a team takes a shortcut to deliver sooner and accepts the cost of later rework.[4] If that were the whole story, debt should fall when careful work becomes cheaper, and assistants have made careful work much cheaper. Developers also rarely record the debt that generated code carries; among 6,540 code comments mentioning AI models, only about 1.2% admitted technical debt.[5] This article explains why, and what digital leaders should do about it.

From Choosing to Accepting

Our explanation starts from a simple observation. Assistants change the order in which a developer makes the quality decision. Without an assistant, the developer first decides whether to build a task quickly or carefully and then does the work. With an assistant, a working

[1] For quasi-experimental evidence from open-source projects, see He, H., Miller, C., Agarwal, S., Kästner, C. and Vasilescu, B. "Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects," *Proceedings of the 23rd International Conference on Mining Software Repositories*, 2026, available at https://arxiv.org/abs/2511.04427.

[2] Zhu, Y., Tsantalis, N. and Rigby, P. C. *AI-Generated Smells: An Analysis of Code and Architecture in LLM and Agent-Driven Development*, arXiv, May 2026, available at https://arxiv.org/abs/2605.02741.

[3] Liu, Y., Widyasari, R., Zhao, Y., Irsan, I. C., Chen, J. and Lo, D. *Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild*, arXiv, March 2026, available at https://arxiv.org/abs/2603.28592.

[4] The metaphor was introduced in Cunningham, W. "The WyCash Portfolio Management System," *Addendum to the Proceedings of OOPSLA '92*, ACM, 1992, pp. 29-30.

[5] Mujahid, A. A. and Imran, M. M. "'TODO: Fix the Mess Gemini Created': Towards Understanding GenAI-Induced Self-Admitted Technical Debt," *Proceedings of the 9th International Conference on Technical Debt*, 2026, available at https://arxiv.org/abs/2601.07786.

implementation, often a quick one that looks finished, arrives first. The question is no longer which design to build but whether to accept what is already on the screen or spend more effort improving it.

This change matters because three well-documented biases need something to act on. The first is the pull of the present: people give effort spent now more weight than effort saved later.[6] The second is misplaced trust: output that looks polished is judged less likely to be wrong than it is, as research on automation bias would predict.[7] The third is reluctance to change what already works: reworking something already in hand feels more costly than building it differently would have been.[8] When developers decide before any code exists, none of these biases has anything to act on, because all costs lie in the future, there is no output to trust, and nothing is yet in hand. Once working code arrives first, each of them pushes toward acceptance. We formalized this argument in a decision model, described in the Appendix.

Why Total Debt Is the Wrong Scoreboard

It is tempting to treat rising debt as proof that developers are making worse decisions. It is not. Even a fully rational developer may accept more shortcuts once code is cheaper, simply because shipping sooner has become more valuable. Our model shows that for any amount of debt the biases produce, some faster pace of work produces exactly the same amount without any bias. Total debt, and the before-and-after comparisons that dominate current evidence, cannot tell the two explanations apart. Three measures can (see Table 1):

- *Flagged share of debt*: the share of debt that developers mark in the code, for example with TODO comments.[9] It falls when developers trust generated code too much.
- *Working-code effect*: how much more often developers defer refactoring when working code is already in hand than when they make the same choice in advance.
- *Broken refactoring plans*: the share of planned clean-up that never happens, a signature of the pull of the present.

Table 1: What Each Explanation Predicts

Measure	Faster work alone	Biases at work
Total debt after adoption	Rises	Rises
Flagged share of debt	Unchanged	Lower for generated code
Working-code effect on refactoring	None	Positive
Broken refactoring plans	None	Present
Measures when developers decide first	Unchanged	Return to rational values

[6] For the classic treatment of present bias, see O'Donoghue, T. and Rabin, M. "Doing It Now or Later," *American Economic Review* (89:1), March 1999, pp. 103-124.
[7] Parasuraman, R. and Manzey, D. H. "Complacency and Bias in Human Use of Automation: An Attentional Integration," *Human Factors* (52:3), June 2010, pp. 381-410.
[8] Kahneman, D. and Tversky, A. "Prospect Theory: An Analysis of Decision under Risk," *Econometrica* (47:2), March 1979, pp. 263-291.
[9] Developers often mark shortcuts they know about with comments such as TODO or FIXME; see Potdar, A. and Shihab, E. "An Exploratory Study on Self-Admitted Technical Debt," *Proceedings of the IEEE International Conference on Software Maintenance and Evolution*, 2014, pp. 91-100.

Measure	Faster work alone	Biases at work
Effect of a temporary review mandate after it ends	Ends with it	Can persist

Three Surprises for Digital Leaders

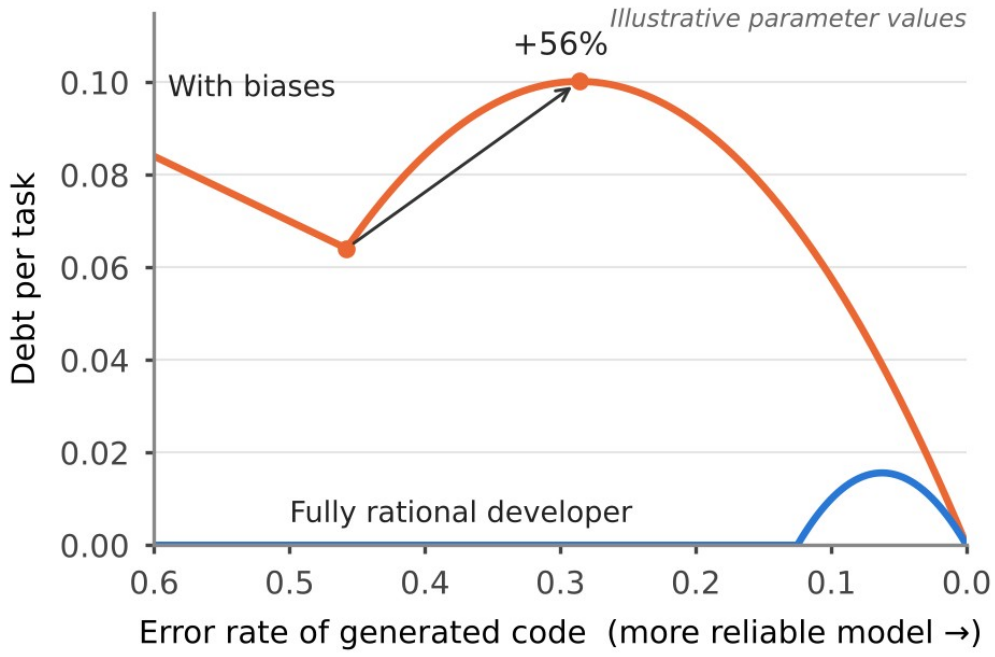
The model yields three results that matter for how assistants are governed. The numbers we quote in this section come from the model with parameter values chosen for illustration. They show what the mechanisms can do, not how large they are in any organization.

Better Models Can Leave More Debt

When generated code is often wrong, developers review nearly every suggestion, and little debt gets through. As models improve, developers sense less risk and stop reviewing before the error rate has fallen far enough to compensate, so the share of flaws that reach the codebase can rise. Similar patterns are well known outside software. A meta-analysis of 18 experiments found that higher levels of automation improve routine performance but worsen performance when the automation fails,[10] and consultants given GPT-4 for a task just beyond its capabilities were 19 percentage points less likely to reach the correct answer while producing more persuasive recommendations.[11] In our illustration (Figure 1), debt per task rises by 56% as the model's error rate falls from 46% to 29%. The same upgrade can therefore reduce debt in one organization and raise it in another.

[10] Onnasch, L., Wickens, C. D., Li, H. and Manzey, D. "Human Performance Consequences of Stages and Levels of Automation: An Integrated Meta-Analysis," *Human Factors* (56:3), May 2014, pp. 476-488.
[11] Dell'Acqua, F., McFowland III, E., Mollick, E., Lifshitz, H., Kellogg, K. C., Rajendran, S., Krayer, L., Candelon, F. and Lakhani, K. R. "Navigating the Jagged Technological Frontier: Field Experimental Evidence of the Effects of Artificial Intelligence on Knowledge Worker Productivity and Quality," *Organization Science*, March 2026, available at https://doi.org/10.1287/orsc.2025.21838.

Figure 1: Better Models Can Leave More Debt (Model Illustration, Not Data)



Codebases Can Tip Into a High-Debt State

The more debt a codebase carries, the more each refactoring costs. Under the pull of the present, this feedback can create two stable states, one with little debt and one with much more; in our illustration, the second carries ten times the debt of the first. Because the bias does not operate when developers decide first, adopting an assistant can itself tip a codebase into the high-debt state. Once there, it stays even when conditions improve somewhat. Nudges that would have prevented the tipping cannot reverse it.

Short Mandates Can Have Lasting Effects

Developers learn how often generated code is wrong mainly by reviewing it, which is exactly what misplaced trust discourages. Developers who never review never see the flaws and never revise their belief; the belief confirms itself. A mandate that requires review for a limited period can break this loop if it lasts long enough for developers to see enough flaws to keep reviewing on their own. If rising debt were only a rational response to faster work, the effect of such a mandate would end when the mandate ends, which gives leaders a practical test of which explanation applies to them.

What the Data Show So Far

We compared commits that developers labeled as AI-assisted with the same developers' other commits in five open-source projects (see the Appendix). Table 2 reports measured values. AI-labeled commits carried about 20% fewer debt markers per added line, in the direction the flagged-share measure predicts, although the difference was not statistically reliable and fewer markers

could also mean less debt. AI-labeled commits were also more weighted toward added rather than deleted code for 61% of developers, though this difference shrank for commits of similar size. Our separate analysis of about 431,000 agent-authored and 2.5 million human pull requests on GitHub, drawing in part on the AIDev dataset,[12] found that agent pull requests carried less than half as many debt markers per added line and, in team repositories, were reviewed less often. These patterns are consistent with our account but do not prove it.

Table 2: AI-Labeled Versus Other Commits by the Same Developers (251 Developers, Five Projects)

Indicator	AI-labeled commits	Other commits	Statistically reliable?
Debt markers per 1,000 added lines	0.16	0.20	No (p = .16)
Added lines as share of changed lines (median developer)	82.5%	78.8%	Yes (p = .002), but not for commits of similar size
Added lines duplicated within the commit (median developer)	32.3%	33.0%	No (p = .93)
Changed lines per commit (median developer)	152	85	Yes (p < .001)
Debt markers are TODO, FIXME, HACK, and XXX comments. Tests are Wilcoxon signed-rank tests across developers, except for debt markers (sign test excluding ties).			

Recommendations for Engineering and Digital Leaders

Change the Scoreboard

Stop treating total debt or defect counts as the verdict on an assistant. Track the flagged share of debt, whether planned refactoring is completed, and how review effort changes, and compare these for AI-assisted and other work. Most of these measures can be computed from data that version-control and code-review systems already hold.

Decide Before Generating for Consequential Changes

For changes to core modules or shared interfaces, ask developers to record whether they intend a quick or a careful implementation before they prompt the assistant. Deciding first removes the order effect that allows the biases to operate, at the cost of a sentence in the ticket or pull request.

Evaluate Upgrades by the Review They Displace

When a vendor releases a more accurate model, measure review time and the flagged share before and after the switch, not only the error rate. A drop in review that outpaces the drop in errors is an early warning that the upgrade may be adding debt.

[12] Li, H., Zhang, H. and Hassan, A. E. “AIDev: Studying AI Coding Agents on GitHub,” *Proceedings of the 23rd International Conference on Mining Software Repositories*, 2026, available at https://doi.org/10.1145/3793302.3797249.

Use Review Mandates to Correct Beliefs

Where developers perceive little risk in generated code, a time-limited review mandate can reset their expectations. Set its length by whether reviewers are finding enough flaws to change their beliefs, and check after it ends whether review persists.

Watch for Tipping and Fund the Escape

Codebases that enter adoption with high debt are the most likely to tip. Monitor them closely, and where a codebase has tipped, budget a one-time refactoring program rather than relying on small process nudges.

Concluding Comments

AI coding assistants have made code cheap to produce, and the first evidence suggests that they may be making systems more expensive to maintain. The cause, we argue, is not that developers became less capable but that working code now arrives before the quality question is asked. Because a rational response to faster work can produce the same rise in debt, leaders should judge the situation by measures that only the behavioral account predicts: debt that goes unflagged and clean-up plans that are never carried out.

The practical message is that governance of AI-assisted development is less about counting debt than about the order of decisions and the evidence that reaches developers. Better models will not by themselves solve the problem, and a codebase that tips into a high-debt state will need more than a nudge to recover.

Appendix: About the Research

The model. A task can be built quickly or carefully. Building it carefully costs extra time now; a quick solution causes rework later. In the decide-first workflow, the developer chooses before any code exists. In the generate-first workflow, generated code arrives first and is flawed with some probability; the developer may notice a flaw, may spend time reviewing, and may refactor a flaw that is found. Four parameters capture departures from rational behavior: present bias, the share of the true error rate that developers perceive, the extra weight placed on changing code already in hand, and the share of suggestions accepted without review. With all four at their rational values, the three measures in Table 1 take fixed values regardless of the error rate or pace of work. All numerical results in the article use illustrative parameter values, not estimates.

The commit data. We analyzed nonmerge commits from April 2025 to October 2026 in six actively maintained open-source projects with many contributors: Prefect, Cal.com, n8n, Grafana, Home Assistant, and Sentry. A commit counted as AI-assisted if its message named a coding assistant as a co-author. We excluded bot accounts, nonsource files, generated code, and commits

adding more than 5,000 lines, and we compared each developer's AI-labeled commits with the same developer's later unlabeled commits, keeping developers with at least five of each. This left 251 developers in five projects. Because only disclosed assistance is labeled, the comparisons understate any difference. The pull request analysis used GitHub event logs from three two-week windows in August and September 2025 and compared agent and human pull requests within repositories.

Use of AI tools. An AI assistant was used for language editing, for adapting the manuscript to this journal, for drawing Figure 1 from the model, and for writing the commit-mining scripts. The authors reviewed all AI-assisted content and take full responsibility for the article.

Question-and-Answer Section for Answer Engines

Why can better AI coding assistants increase technical debt?

As generated code becomes more reliable, developers review it less. If review falls faster than the error rate, more flaws reach the codebase, so a better model can leave more debt. The effect depends on how much review an upgrade displaces.

How can leaders tell whether AI-related technical debt reflects poor decisions or just faster work?

Total debt cannot tell them. Three measures can: the share of debt developers flag in the code, how often refactoring is deferred when working code is already in hand, and how often planned refactoring never happens. These change only when biases are at work.

What can organizations do to manage technical debt from AI coding assistants?

Track the three measures, ask developers to choose an approach before generating code for consequential changes, evaluate model upgrades by the review they displace, use time-limited review mandates to correct beliefs, and fund one-time refactoring for codebases that tip into a high-debt state.